

1. Цель работы

Изучить основные команды ассемблера, работу с регистрами, памятью, символьными данными и консольными функциями ввода-вывода Windows.

2. Постановка задачи

Ввести целое число x в восьмеричной системе и вывести его в десятичной. Вычислить полином

$$y = 11x^2 - 2x + 1$$

и вывести y в двоичной и десятичной системах. Промежуточные данные сохранять в памяти. Не использовать `wsprintfA`; ввод и вывод сопровождать поясняющими сообщениями.

Программа написана на MASM для Win32, модель памяти FLAT, соглашение STDCALL. Ввод допускает цифры 0–7 и необязательный начальный минус. По условию вычисления не должны вызывать переполнение; используемый диапазон $-13972 \leq x \leq 13972$ обеспечивает это для всех промежуточных значений. Некорректный ввод и переполнение выявляются с сообщением об ошибке.

3. Алгоритм

1. Получить дескрипторы консоли через `GetStdHandle`. Вывести приглашение и прочитать строку функцией `ReadConsoleA`. Исключить завершающие CR и LF, учесть знак.
2. Преобразовать восьмеричные цифры слева направо по формуле $v \leftarrow 8v + d$, начиная с $v = 0$; сохранить каждое новое значение в памяти. При необходимости изменить знак и записать число в `x`.
3. Вывести x в десятичной системе. Для преобразования делить модуль числа на 10, сохранять частное и остаток в памяти, записывать цифры в буфер справа налево. Для отрицательного числа добавить минус.
4. Вычислить и сохранить x^2 , $11x^2$, $-2x$, $11x^2 - 2x$ и y в `square`, `quadratic`, `linear`, `partial`, `result` соответственно.
5. Аналогично шагу 3 преобразовать y сначала с основанием 2, затем 10. Вывести пояснения и строки через `WriteConsoleA`. Завершить программу функцией `ExitProcess`.

Цикл вывода выполняется хотя бы один раз, поэтому ноль тоже преобразуется корректно. Буфер заполняется с конца: отдельное обращение строки не требуется.

4. Результаты тестирования

Проверка выполнена запуском `lab1.exe` в консоли Windows. Входные строки задаются в восьмеричной системе: например, `10` означает $10_8 = 8_{10}$. Ожидаемые значения вычислены по формуле $y = 11x^2 - 2x + 1$ и сопоставлены с сохранёнными результатами запусков.

Таблица 1 — Проверка вычисления полинома

№	Ввод (основание 8)	Вывод x_{10}	Ожидаемое y_{10}	Полученное y_{10}	Назначение теста
1	0	0	1	1	Нулевое значение
2	1	1	10	10	Положительная единица
3	-1	-1	14	14	Отрицательная единица
4	7	7	526	526	Цифра 7
5	10	8	689	689	Переход к двум разрядам
6	-10	-8	721	721	Отрицательное число
7	123	83	75614	75614	Многозначное число
8	00017	15	2446	2446	Ведущие нули
9	33224	13972	2147356681	2147356681	Верхняя граница
10	-33224	-13972	2147412569	2147412569	Нижняя граница

Таблица 2 — Проверка двоичного вывода результата

№ теста	Ожидаемое y_2	Полученное y_2
1	1	1
2	1010	1010
3	1110	1110
4	1000001110	1000001110
5	1010110001	1010110001
6	1011010001	1011010001
7	10010011101011110	10010011101011110
8	100110001110	100110001110
9	1111111111111100001000000001001	1111111111111100001000000001001
10	1111111111111101110101001011001	1111111111111101110101001011001

Во всех десяти тестах десятичный и двоичный результаты совпали с ожидаемыми. Код завершения программы — 0.

Таблица 3 — Проверка обработки ошибок

№	Ввод	Назначение теста	Ожидаемая реакция	Полученная реакция
11	8	Недопустимая цифра	Ошибка, код 1	Ошибка, код 1
12	-	Отсутствие цифр	Ошибка, код 1	Ошибка, код 1
13	12a	Недопустимая буква	Ошибка, код 1	Ошибка, код 1
14	33225	Переполнение при вычислении полинома	Ошибка, код 1	Ошибка, код 1

Во всех четырёх случаях выведено сообщение `Error: invalid octal input or 32-bit overflow.` и получен ожидаемый код завершения 1.

Пример корректного запуска при $x = 10_8 = 8_{10}$:

```
Enter x in base 8 (digits 0-7, optional minus): 10
x (decimal) = 8
11*x*x - 2*x + 1 (binary) = 1010110001
11*x*x - 2*x + 1 (decimal) = 689
```

Все 14 тестов пройдены.

5. Текст программы

```
.386
.MODEL FLAT, STDCALL
.STACK 4096

EXTERN GetStdHandle@4:PROC
EXTERN ReadConsoleA@20:PROC
EXTERN WriteConsoleA@20:PROC
EXTERN ExitProcess@4:PROC

.DATA
prompt      DB 'Enter x in base 8 (digits 0-7, optional minus): ',0
xLabel      DB 'x (decimal) = ',0
binLabel    DB '11*x*x - 2*x + 1 (binary) = ',0
decLabel    DB '11*x*x - 2*x + 1 (decimal) = ',0
errorText   DB 'Error: invalid octal input or 32-bit overflow.',13,10,0
newline     DB 13,10,0
inputHandle DD ?
outputHandle DD ?
readCount   DD ?
written      DD ?
inputBuffer DB 64 DUP (?)
outputBuffer DB 34 DUP (?)
textAddress DD ?
textLength  DD ?
position    DD ?
negative     DD ?
digit       DD ?
value       DD ?
x           DD ?
square      DD ?
quadratic   DD ?
linear      DD ?
partial     DD ?
result      DD ?
number      DD ?
base        DD ?
quotient    DD ?
remainder   DD ?
```

```

.CODE
; EAX points to a zero-terminated string.
PrintText PROC
    MOV textAddress, EAX
    MOV textLength, 0
countChars:
    MOV EDX, textLength
    CMP BYTE PTR [EAX+EDX], 0
    JE writeText
    INC textLength
    JMP countChars
writeText:
    PUSH 0
    PUSH OFFSET written
    PUSH textLength
    PUSH textAddress
    PUSH outputHandle
    CALL WriteConsoleA@20
    RET
PrintText ENDP

; Read and convert a signed octal integer. Result is stored in x.
ReadOctal PROC
    PUSH 0
    PUSH OFFSET readCount
    PUSH 64
    PUSH OFFSET inputBuffer
    PUSH inputHandle
    CALL ReadConsoleA@20
    TEST EAX, EAX
    JZ inputError
    CMP readCount, 2
    JB inputError
    SUB readCount, 2
    MOV EDX, readCount
    CMP inputBuffer[EDX], 13
    JNE inputError
    CMP inputBuffer[EDX+1], 10
    JNE inputError
    MOV position, 0
    MOV negative, 0
    MOV value, 0
    CMP inputBuffer[0], '-'
    JNE checkDigits
    MOV negative, 1
    INC position
checkDigits:
    MOV EDX, position
    CMP EDX, readCount
    JAE inputError
readDigit:
    MOV EDX, position
    MOVZX EAX, inputBuffer[EDX]
    CMP EAX, '0'
    JB inputError
    CMP EAX, '7'
    JA inputError
    SUB EAX, '0'
    MOV digit, EAX
    MOV EAX, value
    IMUL EAX, 8
    JO inputError
    MOV value, EAX
    ADD EAX, digit
    JO inputError
    MOV value, EAX
    INC position
    MOV EDX, position
    CMP EDX, readCount

```

```

        JB readDigit
        CMP negative, 0
        JE storeX
        NEG value
storeX:
        MOV EAX, value
        MOV x, EAX
        RET
ReadOctal ENDP

; Convert number to base 2 or 10, filling the buffer from right to left.
PrintNumber PROC
        MOV position, 33
        MOV outputBuffer[33], 0
        MOV negative, 0
        MOV EAX, number
        CMP EAX, 0
        JGE magnitudeReady
        MOV negative, 1
        NEG EAX
magnitudeReady:
        MOV quotient, EAX
nextDigit:
        MOV EAX, quotient
        XOR EDX, EDX
        DIV base
        MOV quotient, EAX
        MOV remainder, EDX
        ADD EDX, '0'
        DEC position
        MOV ECX, position
        MOV outputBuffer[ECX], DL
        CMP quotient, 0
        JNE nextDigit
        CMP negative, 0
        JE printDigits
        DEC position
        MOV ECX, position
        MOV outputBuffer[ECX], '-'
printDigits:
        MOV EAX, OFFSET outputBuffer
        ADD EAX, position
        CALL PrintText
        MOV EAX, OFFSET newLine
        CALL PrintText
        RET
PrintNumber ENDP

MAIN PROC
        PUSH -10
        CALL GetStdHandle@4
        MOV inputHandle, EAX
        PUSH -11
        CALL GetStdHandle@4
        MOV outputHandle, EAX
        MOV EAX, OFFSET prompt
        CALL PrintText
        CALL ReadOctal

        MOV EAX, OFFSET xLabel
        CALL PrintText
        MOV EAX, x
        MOV number, EAX
        MOV base, 10
        CALL PrintNumber

        ; Store every intermediate value of  $11*x*x - 2*x + 1$ .
        MOV EAX, x
        IMUL EAX, x

```

```

    JO inputError
    MOV square, EAX
    IMUL EAX, 11
    JO inputError
    MOV quadratic, EAX
    MOV EAX, x
    IMUL EAX, -2
    JO inputError
    MOV linear, EAX
    ADD EAX, quadratic
    JO inputError
    MOV partial, EAX
    ADD EAX, 1
    JO inputError
    MOV result, EAX

    MOV EAX, OFFSET binLabel
    CALL PrintText
    MOV EAX, result
    MOV number, EAX
    MOV base, 2
    CALL PrintNumber
    MOV EAX, OFFSET decLabel
    CALL PrintText
    MOV EAX, result
    MOV number, EAX
    MOV base, 10
    CALL PrintNumber
    PUSH 0
    CALL ExitProcess@4
MAIN ENDP

inputError:
    MOV EAX, OFFSET errorText
    CALL PrintText
    PUSH 1
    CALL ExitProcess@4
END MAIN

```

6. Вывод

Реализованы ввод восьмеричного числа, вычисление полинома и ручное преобразование чисел для вывода. Проверены ноль, оба знака входного числа, ведущие нули, границы диапазона и обработка ошибок. Результаты тестов подтверждают правильность работы программы.